

Arbre Binaire de Recherche

Parcours, Recherche et Améliorations

Parcours infixe, préfixe et postfixe — Implémentation orientée objet en Python

Module	Structures de données
Langage utilisé	Python (Programmation Orientée Objet)
Bibliothèques de structure de données	Aucune (classes implémentées manuellement)
Date	_____

Réalisé par :

- [Nom Étudiant 1]
- [Nom Étudiant 2]
- [Nom Étudiant 3]
- [Nom Étudiant 4]
- [Nom Étudiant 5]

1. Répartition des tâches

Le tableau ci-dessous présente la répartition du travail entre les membres du groupe. Les noms et tâches doivent être complétés selon l'organisation réelle de l'équipe.

Étudiant	Tâche(s) réalisée(s)
[Nom Étudiant 1]	Implémentation de la classe Noeud, de l'insertion et de la recherche
[Nom Étudiant 2]	Implémentation des parcours infixe et préfixe
[Nom Étudiant 3]	Implémentation du parcours postfixe et du parcours par niveau (largeur)
[Nom Étudiant 4]	Améliorations : suppression, hauteur, min/max, vérification d'équilibre
[Nom Étudiant 5]	Affichage graphique en mode texte, menu interactif de démonstration
Tous	Tests, rédaction du rapport et enregistrement de la vidéo

2. Introduction

Ce projet a pour objectif d'implémenter, en programmation orientée objet et sans recourir à des bibliothèques externes de structures de données, un Arbre Binaire de Recherche (ABR). L'accent est mis sur les trois parcours fondamentaux — infixe, préfixe et postfixe — ainsi que sur la recherche d'une valeur. Plusieurs améliorations ont également été ajoutées afin d'enrichir la structure : suppression, parcours par niveau, calcul de la hauteur, vérification de l'équilibre, et affichage graphique en mode texte.

3. Présentation de la structure

Un Arbre Binaire de Recherche (ABR) est une structure de données hiérarchique dans laquelle chaque noeud possède au plus deux enfants (gauche et droite), et respecte la propriété d'ordre suivante : pour tout noeud N , toutes les valeurs du sous-arbre gauche sont strictement inférieures à N , et toutes les valeurs du sous-arbre droit sont strictement supérieures à N . Cette propriété permet des opérations de recherche, d'insertion et de suppression en $O(\log n)$ en moyenne (dans le cas d'un arbre équilibré), et en $O(n)$ dans le pire cas (arbre dégénéré en liste chaînée).

L'implémentation repose sur deux classes Python : **Noeud**, qui encapsule une valeur et des références vers ses enfants gauche et droit, et **ArbreBinaireRecherche**, qui expose l'ensemble des opérations sur l'arbre (insertion, recherche, suppression, parcours, statistiques).

4. Algorithmes de parcours et de recherche

4.1 Recherche

La recherche exploite la propriété d'ordre de l'ABR : à chaque étape, la valeur cherchée est comparée au noeud courant, ce qui permet d'éliminer soit le sous-arbre gauche, soit le sous-arbre droit, sans jamais parcourir toute la structure.

```
def rechercher(self, valeur):
    courant = self.racine
    while courant is not None:
```

```
if valeur == courant.valeur: return True
elif valeur < courant.valeur: courant = courant.gauche
else: courant = courant.droite
return False
```

4.2 Parcours infixe (in-order)

Ordre : sous-arbre gauche → racine → sous-arbre droit. Ce parcours produit les valeurs de l'arbre triées par ordre croissant, ce qui en fait le parcours le plus utilisé pour un ABR.

```
def _parcours(noeud):
    if noeud is not None:
        _parcours(noeud.gauche)
        resultat.append(noeud.valeur)
        _parcours(noeud.droite)
```

4.3 Parcours préfixe (pre-order)

Ordre : racine → sous-arbre gauche → sous-arbre droit. Ce parcours est utile pour dupliquer/sérialiser un arbre, car il permet de reconstruire l'arbre dans le même ordre d'insertion.

4.4 Parcours postfixe (post-order)

Ordre : sous-arbre gauche → sous-arbre droit → racine. Ce parcours est notamment utilisé pour supprimer un arbre en mémoire (les enfants sont toujours traités avant leur parent) ou pour évaluer des expressions arithmétiques représentées sous forme d'arbre.

Exemple : pour l'arbre construit à partir des valeurs [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45], on obtient :

Parcours	Résultat
Infixe	10, 20, 25, 30, 35, 40, 45, 50, 60, 70, 80
Préfixe	50, 30, 20, 10, 25, 40, 35, 45, 70, 60, 80
Postfixe	10, 25, 20, 35, 45, 40, 30, 60, 80, 70, 50
Largeur (BFS)	50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45

5. Améliorations apportées

- **Suppression d'un noeud** — Gère les trois cas classiques : noeud feuille, noeud avec un seul enfant, et noeud avec deux enfants (remplacement par le successeur, c'est-à-dire le minimum du sous-arbre droit).
- **Parcours par niveau (largeur / BFS)** — Utilise une file (deque) pour parcourir l'arbre niveau par niveau plutôt qu'en profondeur.
- **Calcul de la hauteur** — Calcule récursivement la hauteur de l'arbre, utile pour évaluer sa performance.
- **Recherche du minimum et du maximum** — Descend respectivement à l'extrême gauche ou à l'extrême droite de l'arbre, en $O(h)$ où h est la hauteur.
- **Vérification d'équilibre** — Vérifie, pour chaque noeud, que la différence de hauteur entre les sous-arbres gauche et droit ne dépasse pas 1 (critère de type AVL), afin de détecter une dégénérescence de l'arbre.
- **Affichage graphique en mode texte** — Affiche l'arbre pivoté à 90° dans la console, avec des connecteurs visuels, pour faciliter la vérification visuelle de la structure pendant la démonstration.
- **Comptage du nombre de noeuds** — Maintenu en $O(1)$ grâce à un compteur mis à jour à chaque insertion et suppression.

6. Complexité algorithmique

Opération	Cas moyen (arbre équilibré)	Pire cas (arbre dégénéré)
Insertion	$O(\log n)$	$O(n)$
Recherche	$O(\log n)$	$O(n)$
Suppression	$O(\log n)$	$O(n)$
Parcours (infixe/préfixe/postfixe/largeur)	$O(n)$	$O(n)$
Hauteur / Min / Max	$O(\log n)$	$O(n)$

n représente le nombre de noeuds de l'arbre et h sa hauteur. Dans le pire cas (insertion de valeurs déjà triées), l'ABR dégénère en une liste chaînée et la complexité devient linéaire ; c'est pourquoi les arbres auto-équilibrés (AVL, rouge-noir) existent en pratique, bien qu'ils ne soient pas demandés dans ce projet.

7. Démonstration

Une démonstration complète du programme (menu interactif : insertion, recherche, suppression, affichage des trois parcours, parcours par niveau, affichage graphique de l'arbre et statistiques) a été enregistrée sous forme de capture vidéo, jointe à ce rapport en tant que livrable séparé.

8. Conclusion

Ce projet a permis d'implémenter un Arbre Binaire de Recherche complet en programmation orientée objet, en respectant la contrainte de ne pas utiliser de bibliothèque externe de structure de données. Au-delà des trois parcours demandés (infixe, préfixe, postfixe) et de la recherche, plusieurs améliorations ont été ajoutées pour rendre la structure plus complète et plus proche d'une implémentation professionnelle : suppression, parcours par niveau, calcul de hauteur, vérification d'équilibre et visualisation textuelle de l'arbre.